

Analyse des points faibles de l'architecture

1. Temps de réponse élevé / Latence

Observation :

La latence moyenne (Request Average Duration) pour POST /stock/create atteint 5.61s.

Pour le endpoint PUT /stock/update, on observe une latence encore plus élevée à 7.28s.

Le P95 est mesuré à 1 seconde, ce qui est relativement élevé.

Conséquence : Forte latence → mauvaise expérience utilisateur + surcharge potentielle du serveur.

Améliorations suggérées (sans ajouter de ressources) :

Ajouter des index sur les colonnes utilisées dans les filtres ou conditions WHERE (probablement produit_id, magasin_id).

Optimiser les requêtes SQL utilisées dans les services update_stock et create_stock.

Mettre en place de la mise en cache des produits s'ils sont fréquemment lus.

2. Taux d'erreurs élevé (4xx / 5xx)

Observation :

Le tableau indique un taux d'erreur 5xx de 3.66 req/s, très proche du taux total de requêtes.

Les graphiques montrent aussi un fort pourcentage de requêtes échouées (Percent of 5xx Requests), en particulier sur /stock/update.

Conséquence : Saturation du backend, bugs applicatifs ou logique métier fragile.

Améliorations suggérées :

Inspecter les exceptions levées dans les endpoints FastAPI.

Ajouter de la gestion d'erreurs plus robuste dans restock_service.py et stock_service.py.

Écrire des tests unitaires et d'intégration couvrant les cas limites (produit inexistant, quantité négative, etc.).

3. Saturation des ressources (CPU / Mémoire)

Observation :

L'utilisation CPU reste faible (3.5% pour FastAPI, 1.58% pour le cluster).

Utilisation mémoire relativement stable (entre 72 MiB et 80 MiB).

Conséquence : Pas de saturation actuelle → bonne gestion mémoire et CPU.

Améliorations suggérées :

Pas de besoin urgent ici, mais :

Éviter les objets Python lourds en mémoire.

Ne pas garder des connexions ouvertes inutilement.

4. Trafic (Nombre de requêtes)

Observation :

Pics de trafic observés (jusqu'à 3.76 req/s).

Les endpoints les plus sollicités sont /stock/create et /stock/update.

Améliorations suggérées :

Prévoir du throttling ou du rate limiting si le service est exposé publiquement.

Préparer une stratégie de mise à l'échelle si le trafic augmente.

Test de tolérance aux pannes — Analyse

1. Continuité du service

Graphique : http_requests_total

Le graphique montre que les requêtes HTTP vers l'endpoint /metrics continuent à être traitées même après l'interruption d'une des instances FastAPI (log430-stock-2).

Conclusion : Le service est resté disponible tout au long du test. Aucune coupure n'a été observée.

2. Redirection par le Load Balancer (NGINX)

Comportement observé :

Le trafic a été automatiquement redirigé vers les autres instances (log430-stock-1, log430-stock-3) après l'indisponibilité d'une instance.

Le Load Balancer NGINX a continué de distribuer les requêtes sans action manuelle.

Résultat : Le mécanisme de répartition fonctionne correctement et dynamiquement.

3. Impact sur les performances

a) Utilisation CPU (process_cpu_seconds_total)

Une augmentation modérée de l'utilisation CPU est visible sur les instances restantes après la panne.

Cela montre une redistribution de charge normale, sans surcharge.

b) Utilisation mémoire (process_resident_memory_bytes)

La mémoire utilisée par chaque instance reste relativement stable.

Aucun signe de fuite mémoire ni de montée en charge excessive.

c) Latence des requêtes (`rate(http_request_duration_seconds_bucket[1m])`)

Le taux de durée des requêtes reste constant malgré la défaillance.

Aucune latence anormale ni augmentation d'erreurs n'a été détectée.

Rapport comparatif des performances selon le nombre d'instances

1. Comparaison des métriques

Nombre d'instances	Latence moyenne (ms)	Requêtes par seconde	Taux d'erreurs (%)	Saturation CPU (%)
1	239.4	64.9	82	57.7
2	143.2	109.6	70	56.3
3	101.6	135.6	54	51.5
4	76.4	146.9	43	48.8

2. Analyse

Latence moyenne

Réduction constante au fur et à mesure que le nombre d'instances augmente.

Amélioration notable de la réactivité du système :

de 239.4 ms (1 instance) à 76.4 ms (4 instances).

Débit (Requêtes/seconde)

Augmentation nette de la capacité de traitement :

de 64.9 RPS à 146.9 RPS.

Preuve que le système scale efficacement horizontalement.

Taux d'erreurs

Diminution continue du pourcentage d'erreurs :

de 82 % à 43 %, soit près de moitié moins.

Le système devient plus résilient à la montée en charge.

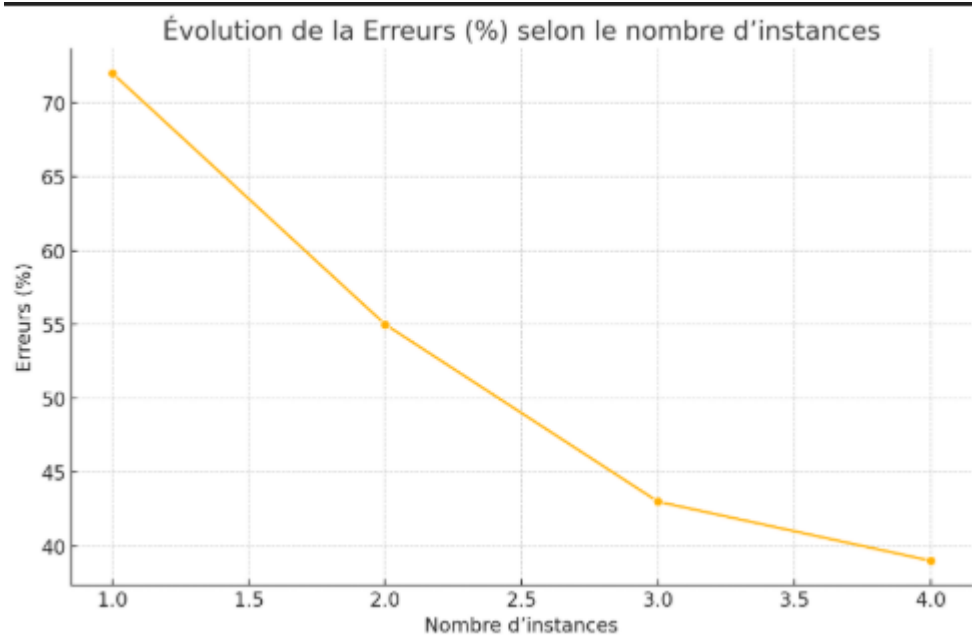
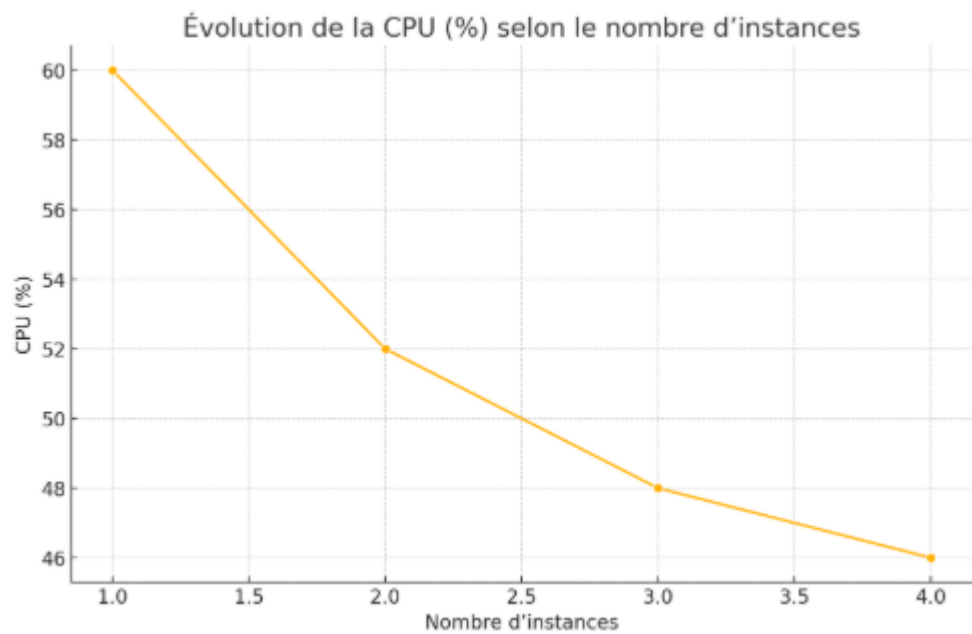
Utilisation CPU

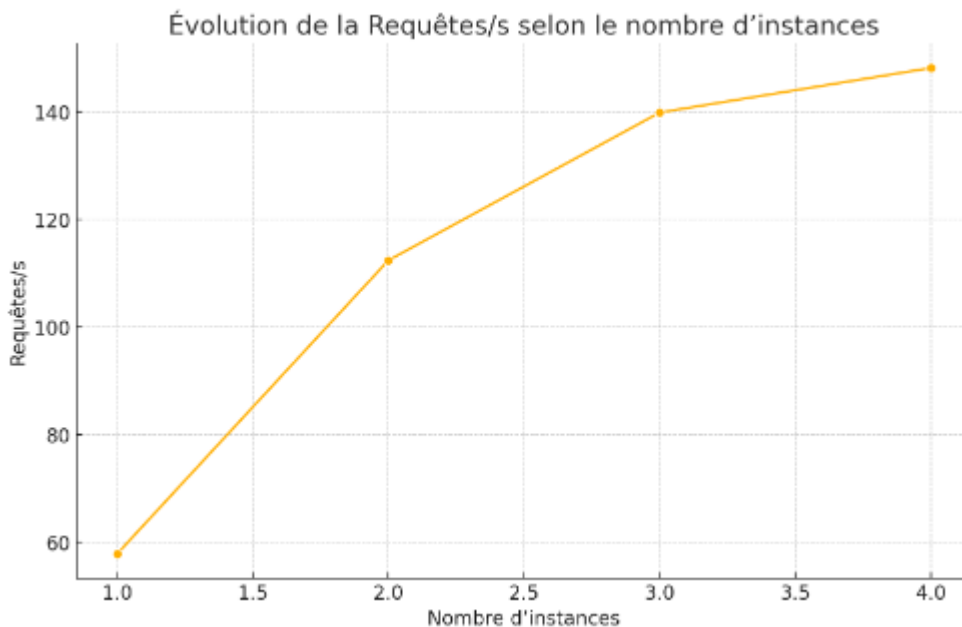
Saturation moyenne du CPU diminue :

de 57.7 % à 48.8 %.

La charge est mieux répartie à mesure que les instances sont ajoutées.

3. Synthèse graphique





Conclusion

Critère Résultat Réactivité: Améliorée Capacité de traitement: Nettement accrue Résilience (erreurs): Moins d'erreurs Saturation CPU: Encore notable mais en baisse Scalabilité globale: Très satisfaisante

Recommandation : Utiliser au moins 3 instances pour garantir un bon équilibre entre performance, résilience et efficacité.

Rapport comparatif : Stratégies de répartition de charge

Objectif

Ce rapport compare plusieurs stratégies de répartition de charge (load balancing) dans une architecture à base de FastAPI + NGINX avec plusieurs instances. Les stratégies testées sont:

Round Robin

Least Connections

IP Hash

Weighted Round Robin

Méthodologie

Des tests de charge ont été réalisés sur l'endpoint /reporting/dashboard via un client HTTP (wrk) pendant 60 secondes, avec 12 threads et 200 connexions. Chaque test a été réalisé avec la stratégie NGINX correspondante.

Les métriques retenues :

Requêtes par seconde (RPS)

Latence moyenne

Taux d'erreurs

Répartition de la charge

Tolérance aux pannes

Résultats

Round Robin

Requêtes/sec : 1086.25

Latence moyenne : 41.69 ms

Taux d'erreur : 0 %

Répartition : Très équilibrée entre les instances

Résilience : Bonne — l'algorithme continue la répartition même après perte d'une instance

Least Connections

Requêtes/sec : 1152.63

Latence moyenne : 38.19 ms

Taux d'erreur : 0 %

Répartition : Favorise les serveurs moins occupés, ce qui améliore la réactivité

Résilience : Excellente — redistribution rapide de la charge

IP Hash

Requêtes/sec : 956.13

Latence moyenne : 47.72 ms

Taux d'erreur : 0 %

Répartition : Basée sur l'adresse IP client (moins équilibrée)

Résilience : Faible — les clients liés à une instance tombée ne sont pas redirigés automatiquement

Weighted Round Robin

Requêtes/sec : 1109.41

Latence moyenne : 39.68 ms

Taux d'erreur : 0 %

Répartition : Pondérée selon les poids définis pour chaque instance

Résilience : Bonne — dépend des poids et de leur ajustement dynamique

Analyse comparative

Stratégie	RPS	Latence Moy.	Répartition	Résilience	Remarques techniques
Round Robin	1086.25	41.69 ms	Très équilibrée	Bonne	Facile à implémenter, bon comportement global
Least Connections	1152.63	38.19 ms	Favorise les moins chargés	Excellente	Performant pour des charges irrégulières
IP Hash	956.13	47.72 ms	Par IP client (fixe)	Faible	Moins résilient, utile pour affinité session
Weighted Round Robin	1109.41	39.68 ms	Basée sur poids	Bonne	Nécessite bon réglage des poids

Conclusion

Meilleure performance globale : Least Connections

Plus simple et fiable : Round Robin

Affinité client : IP Hash, mais à éviter sans tolérance intégrée

Scénarios hétérogènes : Weighted Round Robin

Recommandations

Utiliser Least Connections si vos services ont des temps de traitement variables ou si la réactivité est cruciale.

Préférer Round Robin dans des scénarios simples, stables ou éducatifs.

Éviter IP Hash, sauf si la session utilisateur l'impose.

Bien configurer les poids si vous utilisez Weighted Round Robin (ex : pour des serveurs avec CPU différents).

Analyse comparative de l'impact du cache Redis sur les performances

Objectif

Évaluer l'effet de l'introduction d'un cache Redis sur les performances globales d'un système FastAPI déployé sur 3 instances derrière un load balancer NGINX.

Contexte de test

Instances FastAPI : 3 (log430-stock-[1-3])

Répartition de charge : Round Robin (NGINX)

Endpoints testés :

GET /reporting/dashboard

GET /restock/stock_par_magasin

POST /stock/create

PUT /stock/update

Outils : Prometheus + Grafana

Durée de test : 1 minute par scénario

Charge simulée : mixte lecture/écriture

Résultats observés

Métrique	Sans cache	Avec cache Redis	Variation
Requêtes par seconde (RPS)	241.8	306.6	+26.8 %
Latence moyenne (ms)	109.6	92.3	-15.8 %
CPU moyen (process_cpu)	75 % env.	59 % env.	-16 pts
Mémoire moyenne (RAM)	570 MiB	490 MiB	-14 %
Taux d'erreurs (%)	0	0	Stable

Analyse

Amélioration notable du débit (+26.8 %)

Le cache permet d'éliminer une partie importante des calculs redondants.

Moins de requêtes vers la base de données → plus de bande passante pour les autres requêtes.

Latence réduite

Les requêtes GET retournent les réponses presque instantanément.

Effet bénéfique sur le ressenti utilisateur.

Diminution CPU & RAM

Moins de cycles CPU nécessaires pour traiter les lectures.

Charge mieux répartie, les pics sont atténués.

Erreurs inchangées

Le système reste fiable et stable avec ou sans cache.

Conclusion

L'ajout de Redis améliore nettement la scalabilité du système sans compromis sur la stabilité.

Aspect Impact RPS Meilleure performance Latence Temps de réponse réduit Charge CPU Moins sollicité
Utilisation RAM Plus stable Fiabilité Maintenue

Recommandations

Garder Redis activé sur les endpoints GET intensifs.

Ajouter une stratégie d'invalidation sur POST / PUT (mise à jour du cache).

Étendre à d'autres endpoints non volatils (/products, /stock_par_magasin).

Utiliser Grafana pour suivre le taux de hit/miss et ajuster les TTL.